Documentation d’architecture

Cette documentation sert à décrire les choix d’architecture ainsi que les spécifications de l’API REST.

1. Introduction

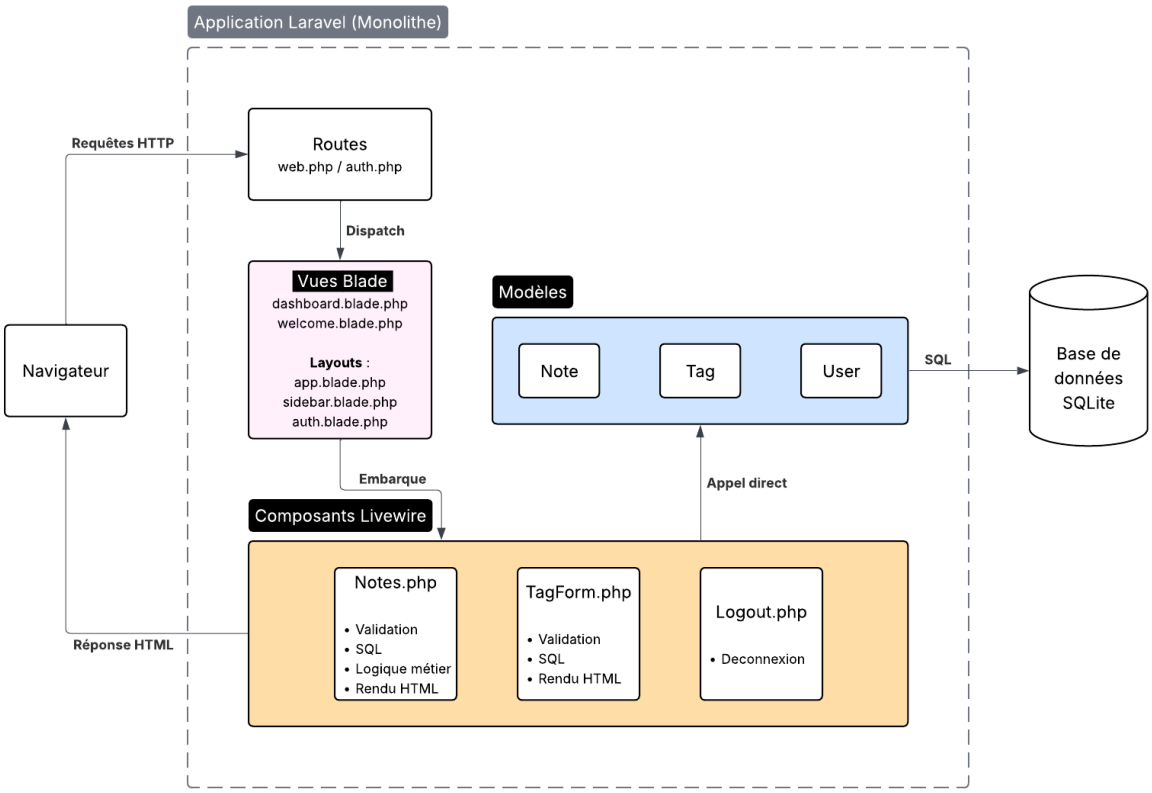
Renote est une application de prises de notes initialement développée sous forme de monolithe server-driven en PHP avec Laravel et Livewire. Dans cette architecture, le serveur générait directement le HTML envoyé au navigateur, sans séparation claire entre la logique métier, l’accès aux données et l’affichage.

Cette refonte a deux objectifs principaux : permettre à l’application de supporter des clients multiples (web et mobile), et améliorer la maintenabilité en appliquant des principes d’architectures reconnus.

La migration s’articule autour des quatre axes suivant :

- **Server-driven → Client-driven :** le front-end devient une application React indépendante qui consomme des données via un API, au lieu de recevoir du HTML généré par le serveur.
- **Passage au MVC + couche service :** le back-end est restructuré en couches distinctes (Routes, Controllers, Services, Repositories, Models), chaque couche ayant une responsabilité unique.
- **Découplage front/back :** les deux parties communiquent exclusivement via une API REST avec un format de réponse uniforme , ce qui les rend indépendantes l’une de l’autre.
- **State management côté front :** Zustand gère l’état de l’application React, centralisant les données et les appels API dans des stores dédiés par domaine métier.

2. Architecture de base



Avantages :

- **Stack unique PHP :** une seule compétence technique requise, cycle de développement rapide
- **Rendu serveur :** HTML généré côté serveur, favorable au SEO et au temps de premier affichage
- **Authentification par session :** pas de gestion de tokens Bearer côté client, pas de configuration CORS

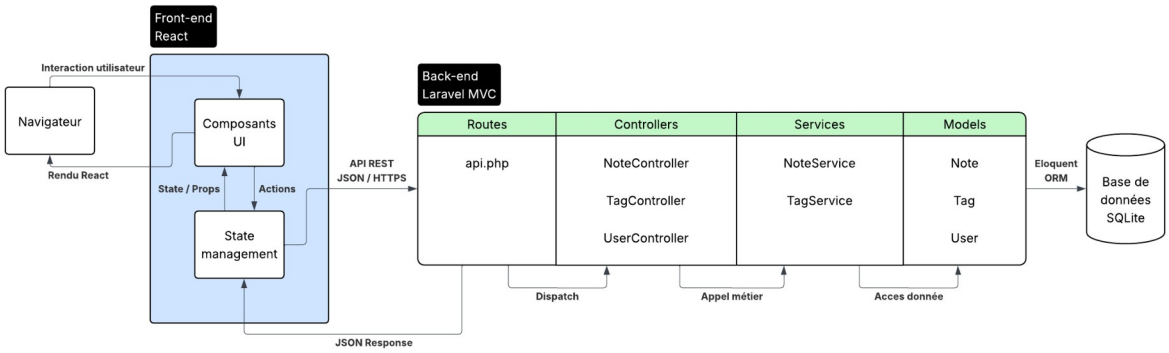
Inconvénients :

- **Violation du Single Responsability Principle :** les composants Livewire melangent validation, logique métier, état et rendu.
- **Couplage fort front/back :** Les composants dépendent directement des modèles. Si on change le model Note, le composant Notes casse. Il n'y a aucune abstraction entre les couches.
- **Absence d'API :** Le serveur renvoie uniquement du HTML. Aucune consomation JSON possible. L'application ne peut pas supporter un client mobile ou un front séparé en l'état.
- **Pas de couche service ou repository :** La logique métier n'est pas isolée, donc non réutilisable et difficilement testable.

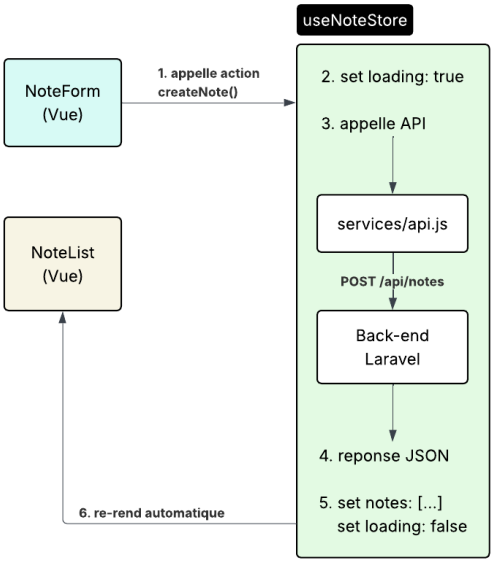
3. Architecture cible

a. Composants d'architecture

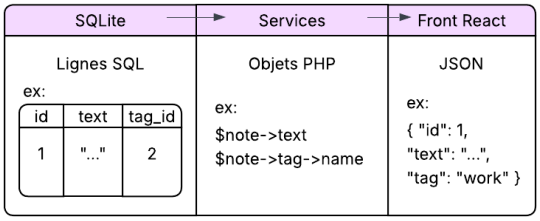
Architecture :



State management :



Nature des données échangées :



b. Spécifications API REST

Conventions

Base URL dev : `http://back.test/api` (Herd) ou `http://localhost:8000/api` (artisan serve)

Format de réponse uniforme :
```json
{
    "status": "success" | "error",
    "message": "Description lisible",
    "data": <payload ou null>
}
```

Authentification : tous les endpoints authentifiés nécessitent le header
```
Authorization: Bearer {token}
```
Token obtenu via `POST /login` ou `POST /register`.

Codes HTTP communs : `200` OK · `201` Créé · `401` Non authentifié · `403` Interdit · `404` Non trouvé · `422` Validation échouée

Authentification (guest)

POST /register
Crée un compte et renvoie un token Bearer.
```json
// Request
{ "name": "Jean", "email": "jean@test.fr", "password": "secret123",
"password_confirmation": "secret123" }

// Response 201
{ "status": "success", "message": "Registration successful", "data":
{ "token": "abc123..." } }
```
Validation : `name` required string ≤255 · `email` required email unique · `password` required min:8 confirmed
Codes : 201 · 422

POST /login
Authentifie et renvoie un token.
```json
// Request
{ "email": "jean@test.fr", "password": "secret123" }

// Response 200
{ "status": "success", "message": "Login successful", "data": { "token":
```

```
"abc123..." } }
```

Codes : 200 · 401 (identifiants invalides) · 422

POST /forgot-password

Envoie un lien de réinitialisation par email.

```json
// Request
{ "email": "jean@test.fr" }

// Response 200
{ "status": "success", "message": "Reset link sent", "data": null }
```

Codes : 200 · 400 (envoi échoué) · 422

POST /reset-password

Réinitialise le mot de passe via token.

```json
// Request
{ "token": "xxx", "email": "jean@test.fr", "password": "newpass123",
"password_confirmation": "newpass123" }

// Response 200
{ "status": "success", "message": "Password reset successfully", "data":
null }
```

Codes : 200 · 400 · 422

Authentifiés (Bearer token requis)

POST /logout

Révoque le token courant.

```json
// Response 200
{ "status": "success", "message": "Logout successful", "data": null }
```

Codes : 200 · 401

GET /profile

Retourne le profil de l'utilisateur courant.

```json
// Response 200
{
    "status": "success",
    "message": "Profile retrieved successfully",
    "data": { "id": 1, "name": "Jean", "email": "jean@test.fr",
"created_at": "2026-04-18T12:00:00Z" }
}
```

Codes : 200 · 401

PUT /profile
Met à jour nom/email.
```json
// Request
{ "name": "Jean-Baptiste", "email": "jb@test.fr" }

// Response 200
{ "status": "success", "message": "Profile updated successfully", "data": {
"id": 1, "name": "Jean-Baptiste", "email": "jb@test.fr", "created_at":
"..." } }
```
Validation : `name` required string ≤255 · `email` required email unique (ignore self)
Codes : 200 · 401 · 422

DELETE /profile
Supprime le compte (nécessite confirmation mot de passe).
```json
// Request
{ "password": "secret123" }

// Response 200
{ "status": "success", "message": "Account deleted successfully", "data":
null }
```
Codes : 200 · 401 · 422

PUT /password
Change le mot de passe.
```json
// Request
{ "current_password": "secret123", "password": "newpass123",
"password_confirmation": "newpass123" }

// Response 200
{ "status": "success", "message": "Password updated successfully", "data":
null }
```
Validation : `current_password` doit matcher · `password` min:8 confirmed
Codes : 200 · 401 · 422

Notes (Bearer token requis)

GET /notes
Liste les notes de l'utilisateur courant.
```json
// Response 200
{
    "status": "success",
```

```
    "message": "Notes retrieved successfully",
    "data": [
        { "id": 1, "text": "Ma note", "tag": { "id": 2, "name": "work" },
"created_at": "...", "updated_at": "..." }
    ]
}
```

Codes : 200 · 401

GET /notes/{id}

Retourne une note appartenant à l'utilisateur.

```json
// Response 200
{ "status": "success", "message": "Note retrieved successfully", "data":
{ "id": 1, "text": "...", "tag": { "id": 2, "name": "work" }, "created_at":
"...", "updated_at": "..." } }
```

Codes : 200 · 401 · 404

POST /notes

Crée une note.

```json
// Request
{ "text": "Ma nouvelle note", "tag_id": 2 }

// Response 201
{ "status": "success", "message": "Note created successfully", "data":
{ "id": 3, "text": "Ma nouvelle note", "tag": { "id": 2, "name": "work" },
"created_at": "...", "updated_at": "..." } }
```

Validation : `text` required string · `tag_id` required exists:tags,id

Codes : 201 · 401 · 422

PUT /notes/{id}

Met à jour une note de l'utilisateur.

```json
// Request
{ "text": "Note modifiée", "tag_id": 3 }

// Response 200
{ "status": "success", "message": "Note updated successfully", "data":
{ "id": 3, "text": "Note modifiée", "tag": { "id": 3, "name": "urgent" },
"created_at": "...", "updated_at": "..." } }
```

Codes : 200 · 401 · 404 · 422

DELETE /notes/{id}

Supprime une note.

```json
// Response 200
```

```
{ "status": "success", "message": "Note deleted successfully", "data": null
}
```

Codes : 200 · 401 · 404

Tags (Bearer token requis)

GET /tags
Liste tous les tags disponibles.

```json
// Response 200
{ "status": "success", "message": "Tags retrieved successfully", "data":
[ { "id": 1, "name": "private" }, { "id": 2, "name": "work" } ] }
```

Codes : 200 · 401

GET /tags/{id}
Retourne un tag.

```json
// Response 200
{ "status": "success", "message": "Tag retrieved successfully", "data":
{ "id": 1, "name": "private" } }
```

Codes : 200 · 401 · 404

POST /tags
Crée un tag.

```json
// Request
{ "name": "urgent" }

// Response 201
{ "status": "success", "message": "Tag created successfully", "data":
{ "id": 4, "name": "urgent" } }
```

Validation : `name` required string ≤50 unique:tags,name
Codes : 201 · 401 · 422

PUT /tags/{id}
Met à jour un tag.

```json
// Request
{ "name": "asap" }

// Response 200
{ "status": "success", "message": "Tag updated successfully", "data":
{ "id": 4, "name": "asap" } }
```

Codes : 200 · 401 · 404 · 422

DELETE /tags/{id}
Supprime un tag.
```json
// Response 200
{ "status": "success", "message": "Tag deleted successfully", "data":
null }
```
Codes : 200 · 401 · 404

c. “Data flow du state management

UI Component → Action → Store → Effet (appel API) → Store → UI Component

1. L’utilisateur déclenche une action

Ex : clic sur "Créer une note" → le composant dispatche une action :
`dispatch(createNote({ text: "Ma note", tag_id: 2 }))`

2. Le store passe en état loading

L’UI affiche un indicateur de chargement.
`{ loading: true, error: null }`

3. L’effet déclenche l’appel API

```
POST /api/notes
Authorization: Bearer {token}
Body: { "text":"Ma note", "tag_id": 2 }
```

4. La réponse met à jour le store

- Succès `201` → La nouvelle note est ajoutée à la liste, `loading` repasse à false
- Erreur → `loading: false, error: "message"`, la liste reste inchangée

5. L’UI se re-rend automatiquement

Les composants abonnés au store reçoivent le nouvel état et se mettent à jour sans action supplémentaire.

Effets gérés côté front

- loading: true/false → affichage d’un spinner pendant les appels API
- error : "message" → affichage d’un message d’erreur en cas d’échec
- Cache local des notes et tags pour éviter des appels redondants

4. Analyse de l’écart

a. Composants à AJOUTER

Back-end :

- routes/api.php → Routes REST
- app/Http/Controllers/NoteControllers.php → CRUD notes
- app/Http/Controllers/TagControllers.php → CRUD nags
- app/Http/Controllers/AuthControllers.php → login / logout
- app/Services/NoteService.php → logique métier notes
- app/Services/TagService.php → logique métier tags
- app/Http/Requests/StoreNoteRequest.php → validation création note
- app/Http/Requests/UpdateNoteRequest.php → validation mise à jour note
- app/Http/Requests/StoreTagRequest.php → validation création tag
- app/Repositories/Interfaces/NoteRepositoryInterface.php
- app/Repositories/Interfaces/TagRepositoryInterface.php
- app/Repositories/NoteRepository.php
- app/Repositories/TagRepository.php

Front-end :

- Composants UI : NoteList, NoteForm, TagForm
- Composants auth : LoginForm, RegisterForm
- Composants settings : ProfileForm, PasswordForm, AppearanceToggle
- Composants layout : AppLayout, Sidebar, Header
- Point d'entrée : App.jsx (router + providers)
- Couche services : api.js (client HTTP centralisé)
- Couche store : stores Zustand

b. Composants à MODIFIER

Back-end :

- app/Models/user.php → ajouter la relation notes() hasMany
- app/Livewire/Notes.php → extraire la logique métier vers NoteService
- app/Livewire/TagForm.php → extraire la logique métier vers TagService

c. Composants à SUPPRIMER

Back-end :

- ressources/views/livewire/notes.blade.php → Remplacé par React
- ressources/views.livewire/tag-form.blade.pgp → Remplacé par React
- app/Livewire/Notes.php → Remplacé par NoteController + NoteService
- app/Livewire/TagForm.php → Remplacé par TagController + TagService
- Vues Volt auth : login, register, forgot-password, reset-password, verify-email
- Vues Volt settings : profile, password, appearance, delete-user-form
- Layouts Blade : dashboard.blade.php, app.blade.php, sidebar.blade.php, auth.blade.php

5. Justification de l'approche architecturale

MVC + couche Service

L'architecture MVC sépare clairement les responsabilités : les Controllers orchestrent les

requêtes, les Services isolent la logique métier, les Models gèrent l'accès aux données. Cette séparation rend chaque couche indépendamment testable et modifiable sans impacter les autres.

Principes SOLID
- **SRP :** chaque classe a une seule responsabilité. Un Controller ne contient pas de logique métier, un Service de fait pas de rendu.
- **OCP :** l'ajout d'un nouveau type de ressource (ex. commentaires) ne nécessite pas de modifier le code existant, seulement d'ajouter de nouveaux fichiers.
- **DIP :** les Services dépendent d'interfaces et non des implémentations concrètes. Cela permet de changer la couche d'accès aux données (ex. passer de SQLite à MySQL) sans modifier la logique métier.

API REST
Le choix d'une API REST permet de découpler complètement le front du back. La même API peut être consommée par un client web, mobile ou un service tiers sans modifier le back-end.

React
Choisi pour son architecture orientée composant qui correspond naturellement à la séparation des responsabilités côté front. Chaque élément de l'UI est un composant indépendant et réutilisable. React est également le framework le plus adopté pour construire des interfaces consommant une API REST et possède un écosystème fourni.

Zustand
<u>Design pattern associé</u> : Flux simplifié + Observer.
- Flux : les données circulent dans un sens unique (Action → Store → Vue), comme Redux, mais sans la séparation formelle actions/reducers/selectors
- Observer : les composants s'abonnent au store via des selectors et sont notifiés automatiquement des changements d'état

<u>Justification :</u>
- Renote est une application à 3 entités, sans interactions de stores complexes. Zustand offre le juste niveau de complexité sans over-engeneering (pas d'optimisation prématurée)
- Code minimal : un create() par store, pas de Provider, pas de reducers. Réduit la surface de maintenance
- Léger et compatible Redux DevTools pour le debug
- En cas de montée en complexité, la migration vers Redux reste possible car le pattern sous-jacent (Flux) est le même

Sécurité
Toutes les routes API sont protégées par un token Bearer. La vérification de propriété des ressources (403 Interdit) empêche un utilisateur d'accéder aux notes d'un autre.